Day 3 – EC2 Instance Creation & Understanding Dependencies

Today's Topic

- What is EC2?
- Dependencies required before creating EC2
- Practical EC2 instance creation
- Understanding networking, storage, and security basics

Introduction to EC2

Amazon EC2 (Elastic Compute Cloud) is a virtual server provided by AWS.

Using EC2, we can:

- Host applications
- Run websites
- Install software
- Create Linux/Windows servers

Before Creating EC2

Before launching an EC2 instance, AWS requires some basic dependencies.

EC2 does not work alone.

To create a server successfully, AWS needs:

- Operating System
- Hardware configuration
- Network
- Firewall
- Storage
- Login authentication

These are called EC2 dependencies.

Step 1 – Login to AWS Console

Open AWS Console:

- Login with AWS account
- Search for EC2
- Open EC2 Dashboard

Flow

User
↓
AWS Console
↓
EC2 Dashboard

Step 2 – Select Region

Choose Region from top-right corner.

Example:

- Mumbai
- Singapore
- Virginia

Why Region?

Region decides:

- Physical location of server
- Latency
- Pricing
- Availability

Example

If application users are in India:
→ Choose Mumbai Region.

Step 3 – Click "Launch Instance"

Now AWS asks for required configurations one by one.

Dependency 1 – Name of EC2

Example:

web-server

Purpose:
Used to identify the instance easily.

Dependency 2 – AMI (Amazon Machine Image)

AMI is the Operating System template.

Examples:

- Amazon Linux
- Ubuntu
- RedHat
- Windows

Without AMI:
→ EC2 cannot boot.

Flow

Physical Machine
↓
Operating System (AMI)
↓
EC2 Instance

Dependency 3 – Instance Type

Instance Type defines:

- CPU
- RAM
- Performance

Example:

t2.micro

Free tier eligible.

Example

- Small application → Small instance
- Large application → Large instance

Diagram

```
Instance Type
  ├── CPU
  ├── RAM
  └── Network
```

Dependency 4 – Key Pair

Key Pair is used for secure login into server.

AWS provides:

- Public Key
- Private Key (.pem file)

Download and store PEM file safely.

Without Key Pair:
→ Cannot login into Linux EC2.

Diagram

PEM File
↓
SSH Login
↓
EC2 Server

Dependency 5 – VPC (Virtual Private Cloud)

VPC is a private network inside AWS.

EC2 instance must be launched inside a VPC.

Without network:
→ Server cannot communicate.

Diagram

AWS Cloud
↓
VPC
↓
EC2 Instance

Dependency 6 – Subnet

Subnet is a smaller network inside VPC.

Types:

- Public Subnet
- Private Subnet

Public subnet:
→ Internet access available

Private subnet:
→ No direct internet access

Diagram

```
VPC
 ├── Public Subnet
 └── Private Subnet
```

Dependency 7 – Public IP

Enable:

Auto Assign Public IP

Public IP allows internet access to EC2.

Without Public IP:

- SSH connection fails
- Website cannot open from internet

Diagram

```
Internet
   ↓
Public IP
   ↓
EC2
```

Dependency 8 – Security Group

Security Group acts as firewall.

It controls traffic entering and leaving EC2.

Important Ports

Port	Purpose
22	SSH
80	HTTP
443	HTTPS

Example

If port 22 is blocked:
→ SSH login will fail.

Diagram

Internet
↓
Security Group
↓
EC2 Instance

Dependency 9 – Storage (EBS)

EC2 requires storage.

AWS attaches EBS volume.

Used for:

- Operating System
- Files
- Applications
- Logs

Default size:

8 GB

Diagram

EC2 Instance
↓
EBS Storage

Final Step – Launch EC2

After configuring all dependencies:
Click:

Launch Instance

AWS now creates:

- Virtual Machine
- Storage
- Network connection
- Firewall rules
- Public IP

Complete EC2 Creation Flow

AWS Account
↓
Select Region
↓
Choose AMI
↓
Choose Instance Type
↓
Create Key Pair
↓
Configure VPC & Subnet
↓
Enable Public IP
↓
Configure Security Group
↓
Attach Storage
↓
Launch EC2 Instance

After Launch

EC2 gets:

- Instance ID
- Public IP

- Private IP
- DNS Name

Status becomes:

Running

Connecting to EC2

Linux command:

ssh -i mykey.pem ec2-user@public-ip

Important Understanding

While launching EC2, AWS internally:

- Creates Virtual Machine
- Allocates CPU & RAM
- Attaches Storage
- Configures Network
- Applies Firewall Rules
- Assigns IP Address
- Boots Operating System

This complete process is called: EC2 Provisioning

Practical – Install NGINX on Ubuntu EC2 Server

Today's Task

- Launch Ubuntu EC2 Instance
- Connect to Ubuntu Server
- Install NGINX
- Configure Security Group
- Create Custom Web Page
- Access Website in Browser

Website Output:

Welcome AItham College Students

Step 1 – Launch Ubuntu EC2 Instance

Open:
[AWS Console]

Go to:

EC2 → Launch Instance

EC2 Configuration

Setting	Value
Name	ubuntu-nginx-server
AMI	Ubuntu Server 24.04
Instance Type	t2.micro
Key Pair	Create new key
Security Group	Allow SSH & HTTP
Storage	8 GB

Important Security Group Rules

Port	Purpose
22	SSH Login
80	HTTP Website Access

Architecture Flow

Browser
↓
Public IP
↓
Security Group
↓
Ubuntu EC2
↓
NGINX Server

Step 2 – Connect to Ubuntu EC2

Copy:

Public IP Address

SSH Command

```
ssh -i mykey.pem ubuntu@public-ip
```

Example:

```
ssh -i mykey.pem ubuntu@13.233.xx.xx
```

Step 3 – Update Ubuntu Packages

Run:

sudo apt update -y

Purpose:
Updates Ubuntu package repository.

Step 4 – Install NGINX

Run:

sudo apt install nginx -y

Purpose:
Installs NGINX web server.

NGINX Installation Flow

Ubuntu Server
↓
Install NGINX
↓
Web Server Ready

Step 5 – Start NGINX Service

Run:

sudo systemctl start nginx

Step 6 – Enable NGINX Service

Run:

sudo systemctl enable nginx

Purpose:
Starts NGINX automatically after reboot.

Step 7 – Verify NGINX Status

Run:

sudo systemctl status nginx

Expected Output:

active (running)

Step 8 – Open NGINX Website Directory

Run:

cd /var/www/html

Step 9 – Create Custom Web Page

Edit index.html:

sudo vi index.html

Press:

i

Paste below HTML:

```
<html>

<head>
<title>AItham College</title>
</head>

<body style="background-color:black; color:white; text-align:center; padding-top:200px;">

<h1>Welcome AItham College Students</h1>

<h2>NGINX Server Running Successfully on Ubuntu EC2</h2>

</body>

</html>
```

Save file:

ESC → :wq

Website Flow

Browser Request
↓
NGINX Service
↓
index.html
↓
Website Displayed

Step 10 – Open Website in Browser

Copy:

EC2 Public IP

Open browser:

http://PUBLIC-IP

Example:

http://13.233.xx.xx

Expected Output

Website displays:

Welcome AItham College Students

NGINX Server Running Successfully on Ubuntu EC2

Complete Architecture

Browser
↓

Internet
↓
Public IP
↓
Security Group (Port 80)
↓
Ubuntu EC2
↓
NGINX
↓
index.html

Important Understanding

When browser accesses Public IP:

1. Request reaches AWS network
2. Security Group checks Port 80
3. EC2 receives request
4. NGINX processes request
5. index.html page displayed

Useful Commands

Restart NGINX

sudo systemctl restart nginx

Stop NGINX

sudo systemctl stop nginx

Check NGINX Status

sudo systemctl status nginx

Real-Time Interview Questions

Question

Why should Port 80 be opened in Security Group?

Answer

Port 80 allows HTTP traffic so users can access the website from browser.